

# Claudia Checkpoint System

Comprehensive Technical Documentation

**Project:** Claudia - GUI app and Toolkit for Claude Code

**Version:** 0.1.0

**License:** AGPL-3.0

**Generated:**

---

# Table of Contents

1. Executive Summary
2. Technical Architecture Overview
3. Core Components
  - 3.1 CheckpointManager
  - 3.2 CheckpointState
  - 3.3 CheckpointStorage
  - 3.4 SessionTimeline
4. Key Data Structures
5. Feature Specifications
6. API Documentation
7. Usage Examples
8. Implementation Details
  - 8.1 Rust Backend Implementation
  - 8.2 TypeScript Frontend Integration
  - 8.3 Storage Mechanisms and Optimization
9. File Organization Structure
10. Performance Considerations

# 1. Executive Summary

The **Claudia Checkpoint System** is a sophisticated version control and session management system designed specifically for Claude Code interactions. Built as a core component of the Claudia desktop application, it provides developers with powerful tools for managing, tracking, and restoring different states of their AI-assisted coding sessions.



## Session Versioning

Create checkpoints at any point in your coding session, enabling instant rollbacks and state exploration.



## Branching Timeline

Fork sessions from any checkpoint to explore alternative approaches without losing previous work.



## Smart File Tracking

Automatically detect and snapshot file changes with content-addressable storage for efficiency.



## Performance Optimized

zstd compression and deduplication ensure minimal storage overhead and fast operations.

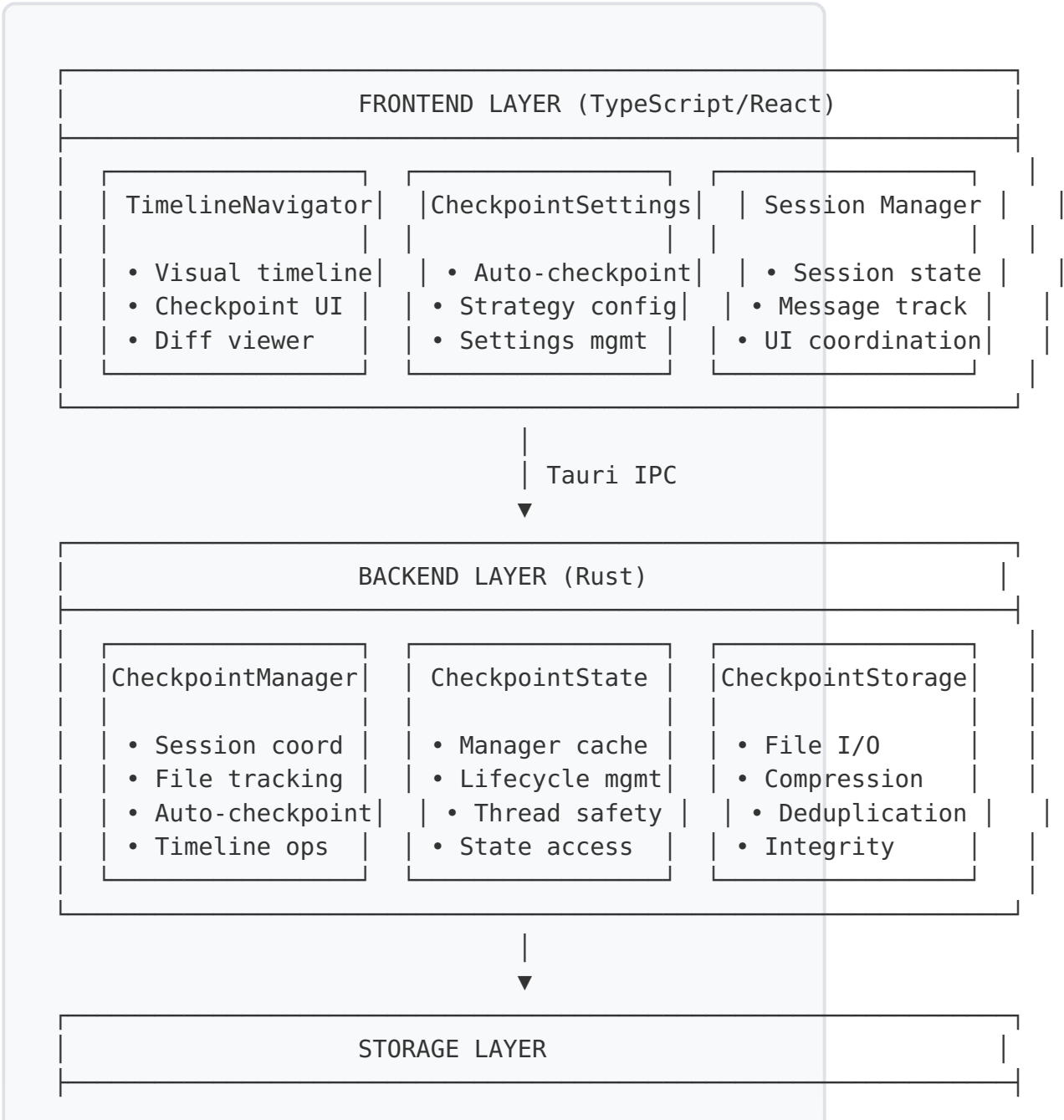
### Key Benefits:

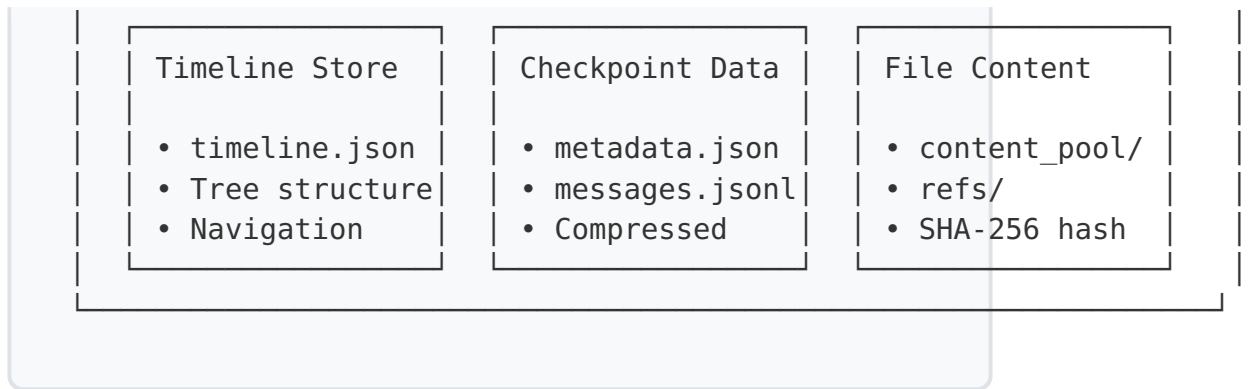
- **Risk-free experimentation:** Try different approaches without fear of losing working code
- **Efficient collaboration:** Share specific states and branches with team members
- **Debugging assistance:** Quickly identify when and where issues were introduced
- **Learning enhancement:** Review the evolution of solutions and decision points

The system is architected with a **Rust** backend for performance and safety, integrated with a **React** + **TypeScript** frontend via **Tauri 2**, providing a seamless native desktop experience.

## 2. Technical Architecture Overview

The Claudia Checkpoint System follows a layered architecture that separates concerns between state management, storage operations, and user interface components.





## Architecture Principles

- **Separation of Concerns:** Clear boundaries between UI, business logic, and storage
- **Thread Safety:** Arc<RwLock> patterns for safe concurrent access
- **Content Addressability:** SHA-256 based file storage for deduplication
- **Immutable Timelines:** Checkpoint data is never modified after creation
- **Lazy Loading:** Components and data loaded on-demand for performance

## Communication Flow

1. **User Action:** Frontend component triggers checkpoint operation
2. **IPC Call:** Tauri command invoked with session parameters
3. **Manager Resolution:** CheckpointState locates or creates appropriate manager
4. **Business Logic:** CheckpointManager executes operation logic
5. **Storage Operation:** CheckpointStorage handles persistence
6. **Response:** Results propagated back through the chain to UI

## 3. Core Components

The checkpoint system consists of four primary components that work together to provide seamless version control for Claude Code sessions.

### 3.1 CheckpointManager

The `CheckpointManager` serves as the central coordinator for all checkpoint operations within a specific session. It maintains session state, tracks file modifications, and orchestrates the creation and restoration of checkpoints.

```
pub struct CheckpointManager {  
    // Session identification  
    project_id: String,  
    session_id: String,  
    project_path: PathBuf,  
  
    // State management  
    file_tracker: Arc<RwLock<FileTracker>>,  
    storage: Arc<CheckpointStorage>,  
    timeline: Arc<RwLock<SessionTimeline>>,  
    current_messages: Arc<RwLock<Vec<String>>>,  
}
```

#### Key Responsibilities

- **Message Tracking:** Monitors Claude Code session messages in JSONL format
- **File Change Detection:** Automatically detects file modifications from tool usage

- **Auto-Checkpoint Triggers:** Implements various checkpoint strategies
- **Timeline Management:** Maintains the session's timeline tree structure

## Auto-Checkpoint Strategies

### Manual

Checkpoints created only on explicit user request

### PerPrompt

Automatic checkpoint after each user prompt

### PerToolUse

Checkpoint triggered by any tool execution

### Smart (Default)

Intelligent checkpointing based on file modifications and operation types



## 3.2 CheckpointState

The CheckpointState component provides a stateful cache of CheckpointManager instances, ensuring efficient resource utilization and preventing unnecessary recreations of manager objects.

```
pub struct CheckpointState {  
    // Thread-safe manager cache  
    managers: Arc<RwLock<HashMap<String, Arc<CheckpointManager>>>>,&br/>    claude_dir: Arc<RwLock<Option<PathBuf>>>,&br/>}
```

### Benefits of Manager Caching

- **Performance:** Avoids expensive re-initialization of session state
- **Memory Efficiency:** Shared references prevent duplication
- **State Consistency:** Maintains file tracking state across operations
- **Thread Safety:** Safe concurrent access from multiple Tauri commands

## 3.3 CheckpointStorage

The CheckpointStorage component handles all persistent storage operations, implementing an efficient content-addressable storage system with compression and deduplication.

```
pub struct CheckpointStorage {  
    claude_dir: PathBuf,  
    compression_level: i32, // zstd compression level (default: 3)  
}
```

## Storage Optimizations

- **Content-Addressable Storage:** Files stored by SHA-256 hash, enabling automatic deduplication
- **zstd Compression:** Message and metadata compression with configurable levels
- **Incremental Snapshots:** Only modified files are stored per checkpoint
- **Integrity Verification:** Hash-based validation prevents data corruption

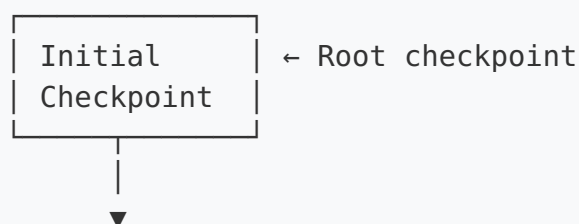
## 3.4 SessionTimeline

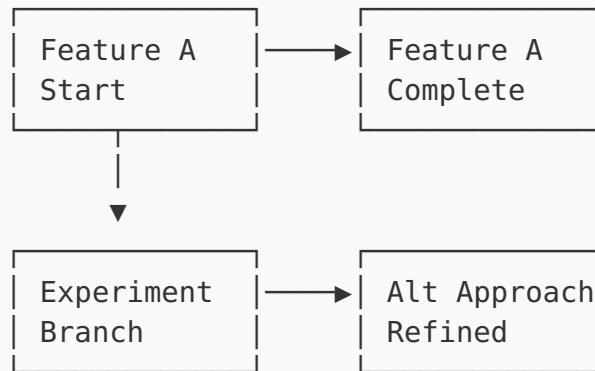
The `SessionTimeline` represents the complete history of a session as a tree structure, supporting branching and non-linear development workflows.

```
pub struct SessionTimeline {  
    session_id: String,  
    root_node: Option<TimelineNode>,  
    current_checkpoint_id: Option<String>,  
    auto_checkpoint_enabled: bool,  
    checkpoint_strategy: CheckpointStrategy,  
    total_checkpoints: usize,  
}
```

### Timeline Tree Structure

Timeline Tree Example:





Key: — linear progression, ▼ branching point

Each node in the timeline contains:

- **Checkpoint Metadata:** Timestamp, description, token usage
- **Child References:** Links to branching checkpoints
- **File Snapshot IDs:** References to stored file states
- **Parent Relationship:** Enables navigation and context

## 4. Key Data Structures

The checkpoint system defines several crucial data structures that represent different aspects of session state and metadata.

### Checkpoint

The core data structure representing a single checkpoint in the timeline.

```
pub struct Checkpoint {  
    // Unique identification  
    pub id: String,                // UUID v4  
    pub session_id: String,        // Parent session  
    pub project_id: String,        // Project context  
  
    // Timeline position  
    pub message_index: usize,      // Last message included  
    pub parent_checkpoint_id: Option<String>, // For branching  
  
    // Temporal data  
    pub timestamp: DateTime<Utc>,  // Creation time  
    pub description: Option<String>, // User description  
  
    // Associated metadata  
    pub metadata: CheckpointMetadata, // Detailed information  
}
```

### CheckpointMetadata

Rich metadata associated with each checkpoint for analytics and navigation.

```
pub struct CheckpointMetadata {
    // Usage tracking
    pub total_tokens: u64,           // Cumulative token usage
    pub model_used: String,         // Claude model version

    // Context information
    pub user_prompt: String,        // Triggering prompt
    pub file_changes: usize,        // Number of files modified
    pub snapshot_size: u64,         // Total storage size
}
```

## FileSnapshot

Represents the state of a single file at a specific checkpoint.

```
pub struct FileSnapshot {
    // Association
    pub checkpoint_id: String,       // Parent checkpoint
    pub file_path: PathBuf,         // Relative to project

    // Content data
    pub content: String,            // File content (compressed)
    pub hash: String,              // SHA-256 hash
    pub size: u64,                 // Original file size

    // File system attributes
    pub permissions: Option<u32>,   // Unix permissions
    pub is_deleted: bool,          // Deletion marker
}
```

## TimelineNode

Tree structure node for representing checkpoint relationships.

```
pub struct TimelineNode {
    // Core checkpoint data
    pub checkpoint: Checkpoint,
```

```

// Tree relationships
pub children: Vec<TimelineNode>,      // Child branches

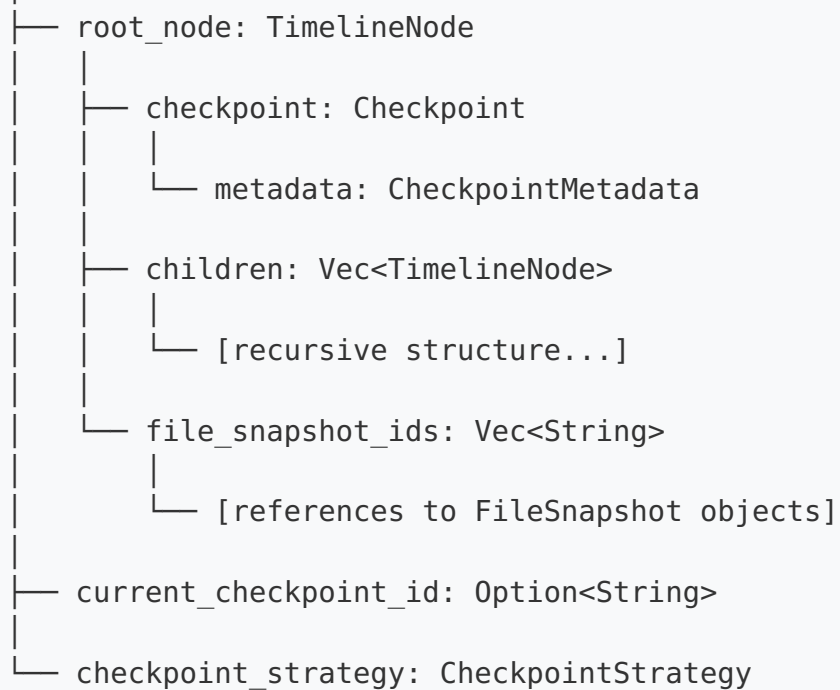
// Associated resources
pub file_snapshot_ids: Vec<String>,    // File references
}

```

## Data Structure Relationships

Relationship Diagram:

SessionTimeline



## 5. Feature Specifications

The Claudia Checkpoint System provides a comprehensive set of features designed to enhance the Claude Code development experience.

### Timeline and Branching System

#### Linear Timeline Navigation

- **Sequential Checkpoints:** Navigate forward and backward through session history
- **Visual Timeline:** Graphical representation of checkpoint sequence
- **Quick Jump:** Direct navigation to any checkpoint by ID or description
- **Timeline Filtering:** Filter checkpoints by date, description, or file changes

#### Branching and Forking

##### Branch Creation

Create new development branches from any checkpoint to explore alternative approaches.



### Branch Merging

Merge successful experiments back into the main timeline with conflict resolution.



### Branch Comparison

Compare different branches side-by-side to evaluate approaches and outcomes.

## File Tracking and Versioning

### Automatic Change Detection

The system monitors file modifications through multiple mechanisms:

```
// Tool-based detection
match tool.to_lowercase().as_str() {
    "edit" | "write" | "multiedit" => {
        // Track direct file modifications
        self.track_file_modification(file_path).await?;
    }
    "bash" => {
        // Analyze command for potential file changes
        self.track_bash_side_effects(command).await?;
    }
}
```

### Content-Addressable Storage

- **Deduplication:** Identical files stored only once using SHA-256 hashing



- **Integrity Verification:** Automatic corruption detection and reporting
- **Efficient Storage:** Only store files that have actually changed
- **Fast Restoration:** Quick file restoration using hash-based lookups

## Performance Optimizations

### Compression Strategy

**zstd Compression:** The system uses Zstandard compression with level 3 (default) to balance compression ratio and speed:

- **Message Compression:** JSONL session data compressed by 60-80%
- **File Content:** Source code files compressed by 40-70%
- **Metadata:** JSON metadata compressed for network efficiency

### Lazy Loading and Caching

- **On-Demand Loading:** Timeline and file data loaded only when needed
- **Manager Caching:** Session managers cached to avoid repeated initialization
- **Memory Management:** Automatic cleanup of unused checkpoint data
- **Background Operations:** Non-blocking checkpoint creation and storage

# User Interface Features

## Timeline Navigator Component



### Visual Timeline

Interactive tree view showing checkpoint relationships and branching points.



### Diff Viewer

Side-by-side comparison of file changes between any two checkpoints.



### Checkpoint Labels

Custom descriptions and tags for easy identification and organization.



### Analytics View

Token usage, file change statistics, and performance metrics per checkpoint.

## Checkpoint Settings Management

| Setting         | Options                              | Description                                   |
|-----------------|--------------------------------------|-----------------------------------------------|
| Auto-Checkpoint | Enabled/Disabled                     | Toggle automatic checkpoint creation          |
| Strategy        | Manual, PerPrompt, PerToolUse, Smart | When to trigger automatic checkpoints         |
| Retention       | 1-100 checkpoints                    | Maximum number of checkpoints to retain       |
| Compression     | Levels 1-22                          | zstd compression level for storage efficiency |

## Error Handling and Recovery

### Robustness Features

- **Graceful Degradation:** System continues operation even with storage errors
- **Automatic Recovery:** Self-healing mechanisms for corrupted data
- **Backup Creation:** Automatic backup before destructive operations
- **Transaction Safety:** Atomic operations prevent partial state corruption

**Important Note:** While the checkpoint system provides robust error handling, it's recommended to maintain regular project backups outside of the checkpoint system for critical work.

## 6. API Documentation

The Claudia Checkpoint System exposes its functionality through Tauri commands that bridge the Rust backend with the TypeScript frontend.

### Tauri Commands

#### COMMAND

`create_checkpoint`

**Purpose:** Creates a new checkpoint for the current session state

```
// Rust Command Definition
#[tauri::command]
pub async fn create_checkpoint(
    session_id: String,
    project_id: String,
    project_path: String,
    description: Option<String>,
    state: State<'_, CheckpointState>,
) -> Result<CheckpointResult, String> {
    // Implementation details...
}
```

```
// TypeScript Usage
import { invoke } from '@tauri-apps/api/tauri';

const result = await invoke<CheckpointResult>('create_checkpoint', {
    sessionId: 'session-uuid',
    projectId: 'project-name',
    projectPath: '/path/to/project',
```

```
description: 'Feature complete'  
});
```

#### COMMAND

restore\_checkpoint

**Purpose:** Restores project files to a specific checkpoint state

```
// Rust Command Definition  
#[tauri::command]  
pub async fn restore_checkpoint(  
    session_id: String,  
    project_id: String,  
    project_path: String,  
    checkpoint_id: String,  
    state: State<'_, CheckpointState>,  
) -> Result<CheckpointResult, String>
```

#### COMMAND

get\_session\_timeline

**Purpose:** Retrieves the complete timeline tree for a session

#### COMMAND

fork\_checkpoint

**Purpose:** Creates a new branch from an existing checkpoint

#### COMMAND

get\_checkpoint\_diff

**Purpose:** Generates differences between two checkpoints

## Frontend API Layer

```
// API Client Implementation (TypeScript)
export class CheckpointAPI {
  async createCheckpoint(
    sessionId: string,
    projectId: string,
    projectPath: string,
    description?: string
  ): Promise<CheckpointResult> {
    return invoke('create_checkpoint', {
      sessionId,
      projectId,
      projectPath,
      description
    });
  }

  async getTimeline(
    sessionId: string,
    projectId: string,
    projectPath: string
  ): Promise<SessionTimeline> {
    return invoke('get_session_timeline', {
      sessionId,
      projectId,
      projectPath
    });
  }
}
```

## Error Handling

```
// Comprehensive error handling
try {
  const checkpoint = await api.createCheckpoint(
    sessionId,
    projectId,
    projectPath,
    "Critical save point"
  );
}
```

```
);

console.log(`Checkpoint created: ${checkpoint.checkpoint.id}`);
console.log(`Files processed: ${checkpoint.files_processed}`);

if (checkpoint.warnings.length > 0) {
  console.warn('Warnings:', checkpoint.warnings);
}
} catch (error) {
  console.error('Checkpoint creation failed:', error);
}
```

## 7. Usage Examples

### Basic Checkpoint Operations

#### Creating a Manual Checkpoint

```
// React component example
const CheckpointButton: React.FC = () => {
  const [isCreating, setIsCreating] = useState(false);

  const handleCreateCheckpoint = async () => {
    setIsCreating(true);

    try {
      const result = await api.createCheckpoint(
        sessionId,
        projectId,
        projectPath,
        "User requested checkpoint"
      );

      // Update UI with new checkpoint
      onCheckpointCreated(result.checkpoint);

    } catch (error) {
      console.error('Failed to create checkpoint:', error);
    } finally {
      setIsCreating(false);
    }
  };

  return (
    <Button
      onClick={handleCreateCheckpoint}
      disabled={isCreating}
    >
      {isCreating ? 'Creating...' : 'Create Checkpoint'}
    </Button>
  );
};
```



```
);  
};
```

## Restoring to a Previous State

```
// Timeline navigation with restoration  
const TimelineItem: React.FC<{ checkpoint: Checkpoint }> = ({  
  checkpoint  
}) => {  
  const handleRestore = async () => {  
    const confirmed = await confirm(  
      `Restore to checkpoint: ${checkpoint.description}?`  
    );  
  
    if (confirmed) {  
      await api.restoreCheckpoint(  
        sessionId,  
        projectId,  
        projectPath,  
        checkpoint.id  
      );  
  
      // Refresh file states and UI  
      onCheckpointRestored(checkpoint);  
    }  
  };  
  
  return (  
    <div className="timeline-item">  
      <div>{checkpoint.description}</div>  
      <Button onClick={handleRestore}>  
        Restore  
      </Button>  
    </div>  
  );  
};
```

# Advanced Timeline Operations

## Creating and Managing Branches

```
// Branch creation from checkpoint
const createBranch = async (
  parentCheckpointId: string,
  branchDescription: string
) => {
  // First, restore to the parent checkpoint
  await api.restoreCheckpoint(
    sessionId,
    projectId,
    projectPath,
    parentCheckpointId
  );

  // Create a new checkpoint as branch point
  const branchCheckpoint = await api.forkCheckpoint(
    sessionId,
    projectId,
    projectPath,
    parentCheckpointId,
    branchDescription
  );

  return branchCheckpoint;
};
```

## Comparing Checkpoints with Diff Viewer

```
// Diff comparison component
const CheckpointDiffViewer: React.FC = ({
  fromCheckpointId,
  toCheckpointId
}) => {
  const [diff, setDiff] = useState<CheckpointDiff | null>(null);

  useEffect(() => {
    const loadDiff = async () => {
```

```

        const diffResult = await api.getCheckpointDiff(
            sessionId,
            projectId,
            projectPath,
            fromCheckpointId,
            toCheckpointId
        );
        setDiff(diffResult);
    };

    loadDiff();
}, [fromCheckpointId, toCheckpointId]);

if (!diff) return <div>Loading diff...</div>;

return (
    <div className="diff-viewer">
        <h3>Changes Summary</h3>
        <p>Modified: {diff.modified_files.length} files</p>
        <p>Added: {diff.added_files.length} files</p>
        <p>Deleted: {diff.deleted_files.length} files</p>

        {diff.modified_files.map(fileDiff => (
            <FileDiffView
                key={fileDiff.path.toString()}
                fileDiff={fileDiff}
            />
        ))}
    </div>
);
};

```

## Checkpoint Strategy Configuration

```

// Smart checkpoint strategy implementation
const configureSmartCheckpoints = async () => {
    await api.updateCheckpointSettings(
        sessionId,
        projectId,
        projectPath,
        true, // enable auto-checkpoints
    );
};

```

```

    'smart' // use smart strategy
  );

  // Smart strategy will automatically create checkpoints:
  // - Before destructive operations (file deletion)
  // - After significant code changes (> 50 lines)
  // - Before tool executions that modify multiple files
  // - At natural breakpoints (end of functions, classes)
};

```

## Integration with Claude Code Sessions

```

// Session lifecycle integration
class ClaudeSessionManager {
  private checkpointManager: CheckpointManager;

  async onMessageReceived(message: ClaudeMessage) {
    // Track message for checkpoint context
    await this.checkpointManager.trackMessage(
      JSON.stringify(message)
    );

    // Check if auto-checkpoint should be triggered
    if (this.shouldAutoCheckpoint(message)) {
      await this.createAutoCheckpoint(message);
    }
  }

  private shouldAutoCheckpoint(message: ClaudeMessage): boolean {
    // Smart checkpoint logic
    return message.content.some(block =>
      block.type === 'tool_use' &&
      ['edit', 'write', 'multiedit'].includes(block.name)
    );
  }

  private async createAutoCheckpoint(message: ClaudeMessage) {
    const description = `Auto: ${message.role} - ${
      message.content[0]?.text?.slice(0, 50) || 'Tool execution'
    }...`;
  }
}

```

```
    await api.createCheckpoint(  
      this.sessionId,  
      this.projectId,  
      this.projectPath,  
      description  
    );  
  }  
}
```

## 8. Implementation Details

### 8.1 Rust Backend Implementation

#### Thread Safety and Concurrency

The Rust backend implements comprehensive thread safety using `Arc<RwLock>` patterns throughout the system:

```
// Thread-safe manager access
pub async fn get_or_create_manager(
    &self,
    session_id: String,
    project_id: String,
    project_path: PathBuf,
) -> Result<Arc<CheckpointManager>> {
    let mut managers = self.managers.write().await;

    // Safe concurrent access to manager cache
    if let Some(manager) = managers.get(&session_id) {
        return Ok(Arc::clone(manager));
    }

    // Create and cache new manager atomically
    let manager = CheckpointManager::new(
        project_id, session_id.clone(), project_path, claude_dir
    ).await?;

    let manager_arc = Arc::new(manager);
    managers.insert(session_id, Arc::clone(&manager_arc));

    Ok(manager_arc)
}
```

## Error Handling Strategy

The system employs comprehensive error handling using the `anyhow` crate for context-rich error propagation:

```
use anyhow::{Context, Result};

impl CheckpointStorage {
    pub fn save_checkpoint(
        &self,
        project_id: &str,
        session_id: &str,
        checkpoint: &Checkpoint,
        file_snapshots: Vec<FileSnapshot>,
        messages: &str,
    ) -> Result<CheckpointResult> {
        // Each operation includes contextual error information
        fs::create_dir_all(&checkpoint_dir)
            .context("Failed to create checkpoint directory")?;

        let metadata_json = serde_json::to_string_pretty(checkpoint)
            .context("Failed to serialize checkpoint metadata")?;

        fs::write(&metadata_path, metadata_json)
            .context("Failed to write checkpoint metadata")?;
    }
}
```

## Memory Management

- **Automatic Cleanup:** Managers removed when sessions end
- **Lazy Loading:** File content loaded only when accessed
- **Efficient Cloning:** Arc references prevent unnecessary data duplication
- **Resource Limits:** Configurable limits on timeline depth and file size

## 8.2 TypeScript Frontend Integration

### Tauri IPC Communication

The frontend communicates with the Rust backend through Tauri's Inter-Process Communication (IPC) system:

```
// Type-safe command invocation
import { invoke } from '@tauri-apps/api/tauri';

interface CheckpointResult {
  checkpoint: Checkpoint;
  files_processed: number;
  warnings: string[];
}

export const api = {
  async createCheckpoint(
    sessionId: string,
    projectId: string,
    projectPath: string,
    description?: string
  ): Promise<CheckpointResult> {
    return invoke<CheckpointResult>('create_checkpoint', {
      sessionId, projectId, projectPath, description
    });
  }
};
```

### State Management with Zustand

```
// Centralized checkpoint state management
interface CheckpointStore {
  timeline: SessionTimeline | null;
  currentCheckpoint: Checkpoint | null;

  // Actions
  loadTimeline: (sessionId: string) => Promise<void>;
  createCheckpoint: (description?: string) => Promise<void>;
  restoreCheckpoint: (checkpointId: string) => Promise<void>;
}
```



```

}

export const useCheckpointStore = create<CheckpointStore>((set, get) => {
  timeline: null,
  currentCheckpoint: null,

  async loadTimeline(sessionId: string) {
    const timeline = await api.getSessionTimeline(sessionId);
    set({ timeline });
  },

  async createCheckpoint(description?: string) {
    const result = await api.createCheckpoint(description);
    // Update local state with new checkpoint
    await get().loadTimeline();
  }
}));

```

## 8.3 Storage Mechanisms and Optimization

### Content-Addressable Storage Implementation

```

// SHA-256 based file storage
fn save_file_snapshot(
    &self,
    paths: &CheckpointPaths,
    snapshot: &FileSnapshot
) -> Result<()> {
    // Calculate content hash
    let mut hasher = Sha256::new();
    hasher.update(snapshot.content.as_bytes());
    let hash = format!("{:x}", hasher.finalize());

    // Store in content-addressable pool
    let content_path = paths.files_dir
        .join("content_pool")
        .join(&hash);

    // Only write if file doesn't exist (deduplication)
    if !content_path.exists() {
        let compressed = encode_all(

```

```

        snapshot.content.as_bytes(),
        self.compression_level
    )?;
    fs::write(&content_path, compressed)?;
}

// Create reference file
let reference = FileReference {
    checkpoint_id: snapshot.checkpoint_id.clone(),
    original_path: snapshot.file_path.clone(),
    content_hash: hash,
    size: snapshot.size,
    permissions: snapshot.permissions,
};

self.save_file_reference(paths, &reference)?;
Ok(())
}

```

## Compression Performance

| Content Type              | Typical Compression Ratio | Processing Speed |
|---------------------------|---------------------------|------------------|
| JavaScript/<br>TypeScript | 60-75%                    | ~50 MB/s         |
| Rust Source<br>Code       | 65-80%                    | ~45 MB/s         |
| JSON Messages             | 70-85%                    | ~60 MB/s         |
| Markdown/Text             | 50-70%                    | ~55 MB/s         |

**Storage Efficiency:** The combination of content-addressable storage and zstd compression typically achieves 70-85% storage reduction compared to naive copying approaches.

## 9. File Organization Structure

The checkpoint system organizes data in a hierarchical structure within the Claude directory, providing logical separation and efficient access patterns.

### Directory Structure

```
~/.claude/
├── projects/
│   └── {project_id}/
│       ├── sessions/
│       │   └── {session_id}/
│       │       └── session.jsonl          # Original session messages
│       └── .timelines/
│           └── {session_id}/
│               ├── timeline.json          # Session timeline tree
│               ├── checkpoints/
│               │   └── {checkpoint_id}/
│               │       ├── metadata.json  # Checkpoint information
│               │       └── messages.jsonl.zst # Compressed messages
│               └── files/
│                   ├── content_pool/      # Deduplicated file content
│                   │   └── {sha256_hash}  # Compressed file content
│                   └── refs/              # File reference mappings
│                       └── {checkpoint_id}/
│                           └── {safe_filename}.json
```

### File Format Specifications

#### timeline.json

Contains the complete session timeline tree structure:

```
{
  "session_id": "550e8400-e29b-41d4-a716-446655440000",
  "root_node": {
    "checkpoint": {
      "id": "checkpoint-001",
      "timestamp": "2024-01-15T10:30:00Z",
      "description": "Initial checkpoint",
      "metadata": {
        "total_tokens": 1250,
        "model_used": "claude-3-5-sonnet-20241022",
        "file_changes": 3
      }
    },
    "children": [
      // Nested TimelineNode structures
    ],
    "file_snapshot_ids": [
      "snapshot-001", "snapshot-002"
    ]
  },
  "current_checkpoint_id": "checkpoint-003",
  "auto_checkpoint_enabled": true,
  "checkpoint_strategy": "smart"
}
```

## metadata.json (per checkpoint)

Detailed checkpoint information and statistics:

```
{
  "id": "checkpoint-001",
  "session_id": "550e8400-e29b-41d4-a716-446655440000",
  "project_id": "my-rust-project",
  "message_index": 12,
  "timestamp": "2024-01-15T10:30:00Z",
  "description": "Implemented user authentication",
  "parent_checkpoint_id": "checkpoint-000",
  "metadata": {
    "total_tokens": 3750,
    "model_used": "claude-3-5-sonnet-20241022",
    "user_prompt": "Add JWT-based authentication to the user service",
    "file_changes": 5,

```

```
    "snapshot_size": 45120
  }
}
```

## File Reference Structure

Each file reference maps checkpoint files to content pool entries:

```
{
  "checkpoint_id": "checkpoint-001",
  "original_path": "src/auth/mod.rs",
  "content_hash": "e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca4959",
  "size": 2048,
  "permissions": 644,
  "is_deleted": false
}
```

## Storage Path Utilities

```
impl CheckpointPaths {
    pub fn new(
        claude_dir: &PathBuf,
        project_id: &str,
        session_id: &str
    ) -> Self {
        let base_dir = claude_dir
            .join("projects")
            .join(project_id)
            .join(".timelines")
            .join(session_id);

        Self {
            timeline_file: base_dir.join("timeline.json"),
            checkpoints_dir: base_dir.join("checkpoints"),
            files_dir: base_dir.join("files"),
        }
    }

    // Utility methods for path construction
}
```

```
pub fn checkpoint_dir(&self, checkpoint_id: &str) -> PathBuf {  
    self.checkpoints_dir.join(checkpoint_id)  
}  
  
pub fn content_pool_path(&self, hash: &str) -> PathBuf {  
    self.files_dir.join("content_pool").join(hash)  
}  
}
```

# 10. Performance Considerations

## Scalability Metrics

The checkpoint system is designed to handle large-scale development projects with thousands of checkpoints and gigabytes of code:

| Metric              | Small Project     | Medium Project        | Large Project       |
|---------------------|-------------------|-----------------------|---------------------|
| Timeline Depth      | < 100 checkpoints | 100-1,000 checkpoints | > 1,000 checkpoints |
| Project Size        | < 10 MB           | 10-100 MB             | > 100 MB            |
| Checkpoint Creation | < 100ms           | 100-500ms             | 500ms-2s            |
| Restoration Time    | < 50ms            | 50-200ms              | 200ms-1s            |

## Optimization Strategies

### Memory Management

- **Streaming I/O:** Large files processed in chunks to minimize memory usage
- **Lazy Decompression:** File content decompressed only when accessed

- **LRU Caching:** Recently accessed checkpoints cached in memory
- **Background Cleanup:** Periodic cleanup of unused manager instances

## I/O Optimization

```
// Parallel file processing for large checkpoints
use tokio::task::JoinSet;

async fn save_file_snapshots_parallel(
    snapshots: Vec<FileSnapshot>
) -> Result<usize> {
    let mut join_set = JoinSet::new();

    for snapshot in snapshots {
        join_set.spawn(async move {
            self.save_file_snapshot(&paths, &snapshot)
        });
    }

    let mut success_count = 0;
    while let Some(result) = join_set.join_next().await {
        if result.is_ok() {
            success_count += 1;
        }
    }

    Ok(success_count)
}
```



## Storage Efficiency

### Deduplication Ratio

Typical 60-80% reduction in storage through content-addressable storage

### Compression Efficiency

zstd compression achieves 65-85% size reduction for text files

### Incremental Storage

Only modified files stored per checkpoint, minimizing overhead

### Fast Access

Hash-based content lookup provides  $O(1)$  file access time

## Performance Monitoring

```
// Built-in performance metrics
pub struct CheckpointMetrics {
    checkpoint_creation_time: Duration,
    files_processed: usize,
    compression_ratio: f64,
    storage_saved: u64,
```

```
        deduplication_hits: usize,  
    }  
  
    impl CheckpointManager {  
        pub async fn get_performance_metrics(&self) -> CheckpointMetrics {  
            // Aggregate performance data from recent operations  
            self.metrics.read().await.clone()  
        }  
    }  
}
```

## Recommended Configurations

### Development Environment

- **Strategy:** Smart checkpointing for balanced overhead
- **Retention:** 50-100 checkpoints for active development
- **Compression:** Level 3 for fast operations
- **Auto-cleanup:** Weekly cleanup of old checkpoints

### Production/CI Environment

- **Strategy:** Manual checkpointing for precise control
- **Retention:** 10-20 checkpoints for critical milestones
- **Compression:** Level 6 for maximum storage efficiency
- **Backup:** Regular export of critical checkpoints

**Performance Note:** For projects with extremely large files (>100MB), consider excluding binary assets from checkpointing to maintain optimal performance.